



University
of Exeter

COMM117

Large Language Models and Application

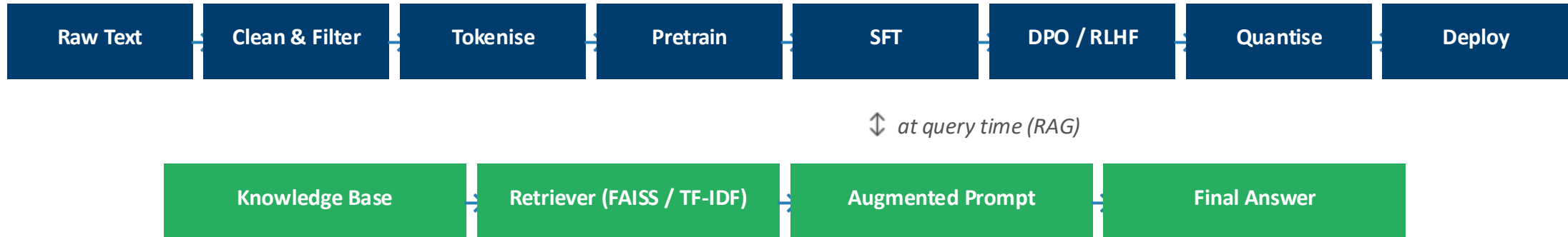
Introduction Today

- Robustness LLM : attack, poisoning and defence
- Advanced LLM efficiency

What We Built

Period	Topic	Key Skills
L 1–3	Data preprocessing & tokenisation	Quality filters, BPE, vocab building
L 4- 6	Transformer architecture	MHA, Causal mask, Pre-LN block
L 7-8	Training at scale	Position encoding, LR schedule, clipping
L 9 -10	Post-training: SFT · DPO · LoRA	Loss masking, preference optimization
L 11-12	LLM API; Text generation	API, sampling, speculative decoding
L 13	Evaluation	BLEU, ROUGE, semantic evaluation, human
L 14-L 15	Summarization and translation	TF-IDF, Text Rank, zero shot prompt, SFT
L 16-L 18	RAG and chaining	Retrieval, Chunking, RAG variants, Evaluation
L 18-L 19	Responsible AI	Biased, toxicity, privacy, fairness
L10	AI robustness and efficiency	Attack, defense, quantization, distillation

The Full LLM Pipeline — All at Once



Data

Why It Matters

- Pretraining is self-supervised — no clean labels
- Model behaviour mirrors data distribution
- Bias, toxicity, factual errors originate here
- "Garbage in, garbage out"

What You Implemented

- **Normalise:** HTML unescape, whitespace, Unicode
- **Remove web artefacts:** URLs, @mentions, #hashtags
- **Language filter:** langdetect
- **Quality heuristics:** length, repetition, symbol ratio
- **Deduplication:** exact-match rate before vs after

Tokenisation

Unit	Pros	Cons	Typical use
Character	No OOV; robust	Very long sequences	Small models; some multilingual
Word	Readable; short seq.	OOV + huge vocab	Classic NLP; not common in LLMs
Subword	Good trade-off	Tokenizer quirks	Most modern LLMs
Byte	Covers any text	More tokens in some cases	Byte-level BPE family

Byte Pair Encoding (BPE)

- Input: **base alphabet** (characters or bytes) + **target** vocabulary size
- Repeatedly **merge** the most frequent neighbor pair
- Each merge **adds a new token** to the vocabulary
- Repeat until you reach the desired vocab size
- After enough merges → common words/subwords become single tokens

WordPiece

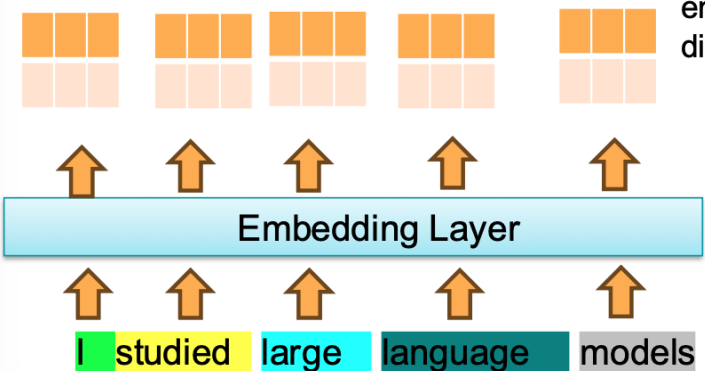
- Choose merges that improve a likelihood objective
→ chooses a symbol pair which will **result in the largest increase in likelihood upon merging**.
- Popularised **by BERT-family** models

Embedding

Embedding in LLM

$$E \in \mathbb{R}^{|V| \times d}$$

D is model
embedding
dimension



Generate Input Embeddings

➡ $PE(p) = [\sin(.), \cos(.), \sin(.), \cos(.), \dots]$ **Absolute PE**

token embeddings → (+ **absolute PE**) → QKV



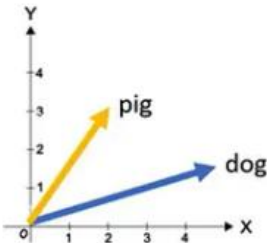
token embeddings → QKV → **Additive positional embedding K, V** → scores



token embeddings → QKV → **apply RoPE to Q, K** → scores

RoPE

The pig chased the dog



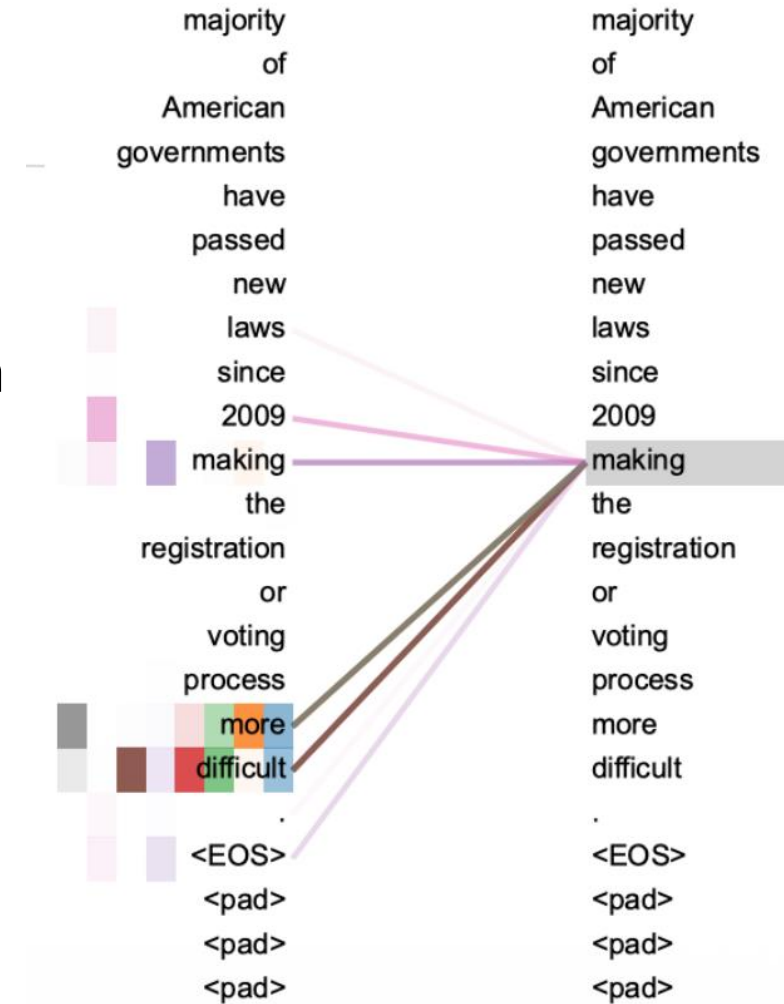
Transformer- Attention

Attention

To find the attention of a target word relative to input embeddings:

- Use the **Query (Q)** of the **target word**.
- Use the **Key (K)** of the **input word**.
- Compute a **matching score (attention weight)** from Q and K.
- Use these scores as weights to sum the Value (V) vectors.

Self attention

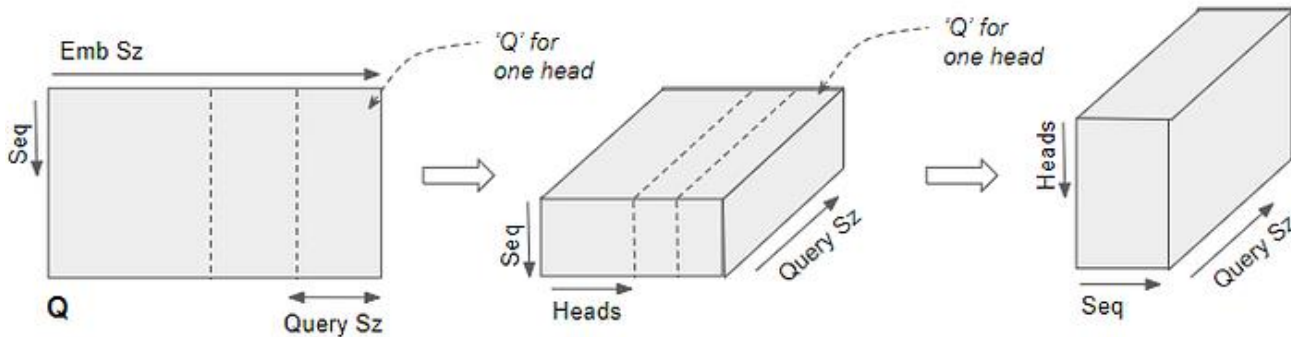


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_h}}\right) \cdot V$$

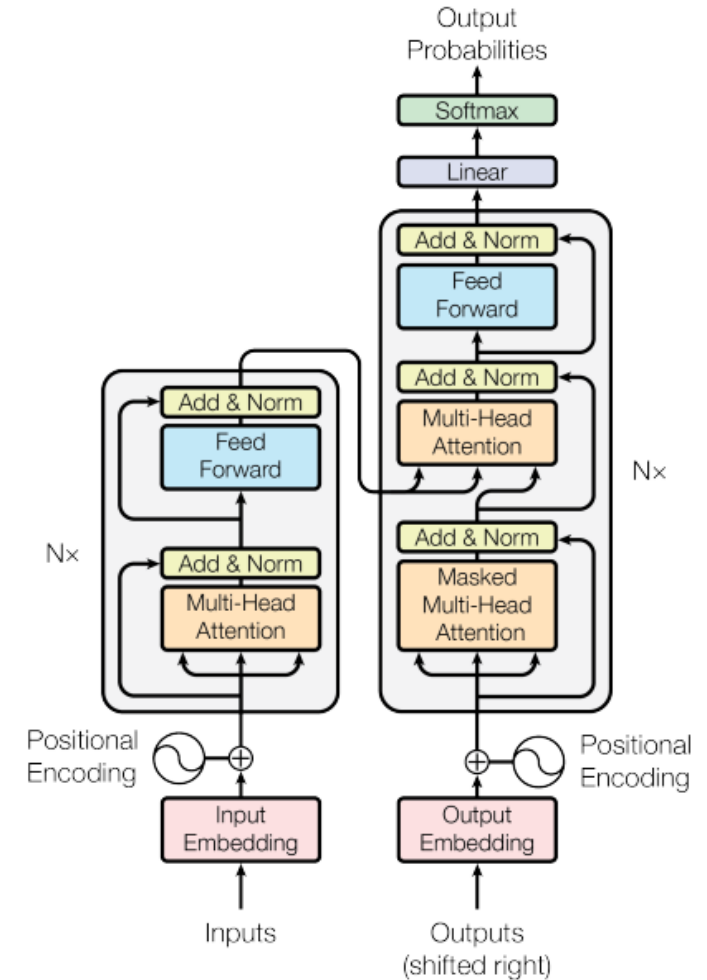
Transformer

Multi-head attention Split into heads!

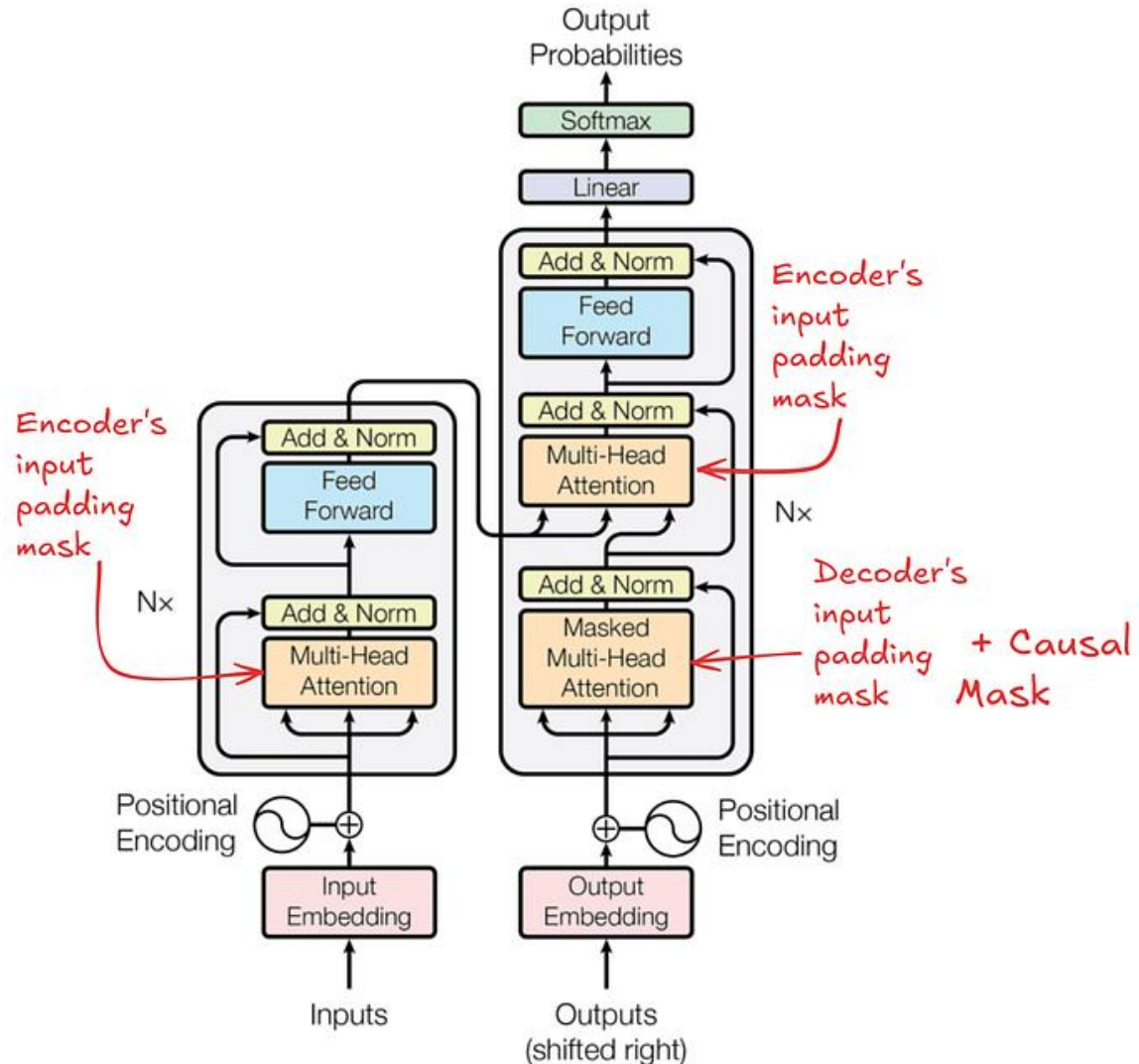
- Reshaping the Q, K, and V matrices
- The Q, K, and V matrices output by the Linear layers are reshaped to include an explicit Head dimension.
- Each 'slice' corresponds to a matrix per head.



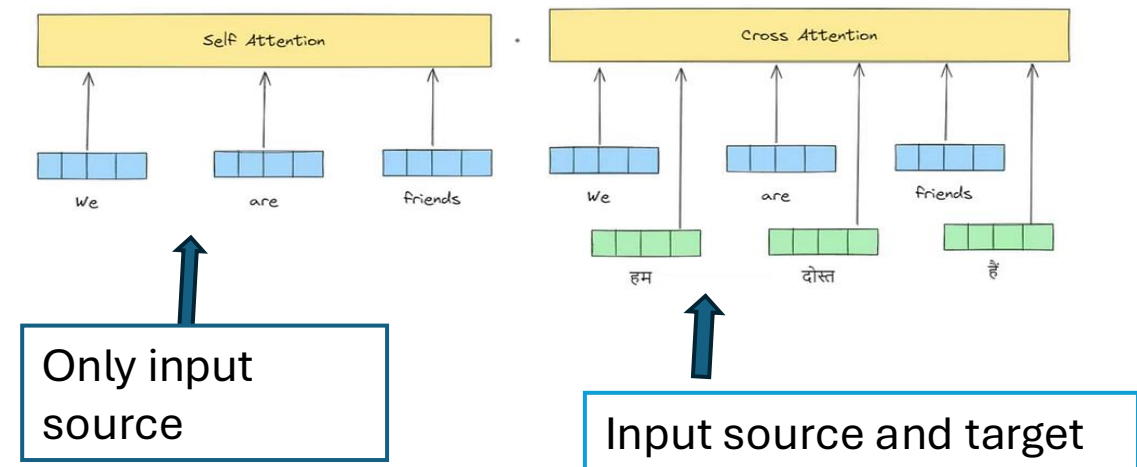
- Attention mixes **information across tokens**.
- The FFN mixes **features within each token**.
- Residual+Norm keep **training stable**



Encoder



- **Encoder-only models (like BERT):** Use a bidirectional approach to understand context from both directions → **bidirectional self-attn**
- **Decoder-only models (like GPT, Llama):** process text from left to right and are particularly good at text generation tasks → **causal self-attn**
- **Encoder-decoder models (like T5, BART):** combine both approaches, using an encoder to understand the input and a decoder to generate output. → **encoder self-attn + decoder masked self-attn + cross-attn**



Objective



BERT (Encoder-only)

Devlin et al., 2018

The cabs ____ the same rates as those ____ by horse-drawn cabs and were ____ quite popular, ____ the Prince of Wales (the ____ King Edward VII) travelled in _____. The cabs quickly ____ known as "hummingbirds" for ____ noise made by their motors and their distinctive black and ____ livery. Passengers ____ the interior fittings were ____ when compared to ____ cabs but there ____ some complaints ____ the ____ lighting made them too ____ to those outside ____.

charged, used, initially, even, future, became, the, yellow, reported, that, luxurious, horse-drawn, were that, internal, conspicuous, cab

Masked token prediction

$$\mathcal{L}_{MLM} = - \sum_{t \in M} \log p_{\theta}(x_t | x_{\setminus M})$$



T5 (Encoder-decoder)

Raffel et al., 2019

Original text

Thank you for inviting me to your party last week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

Denoising span-mask prediction



Decoder-Only

Text: Second Law of Robotics: A robot must obey the orders given it by human beings



Generated training examples

Example #	Input (features)	Correct output (labels)
1	Second law of robotics :	a
2	Second law of robotics : a	robot
3	Second law of robotics : a robot	must
...		

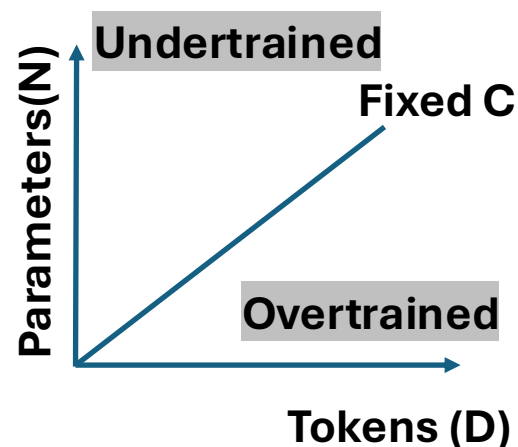
Next Token Prediction

Autoaggressive

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

Training at Scale

Too **few** tokens for model size



Too **many** tokens for model size

Constraints: **memory, throughput, numeric, communication**

LLMs: AdamW + warmup + decay + grad clipping

Optimisation

Training at Scale

MIXED PRECISION TRAINING

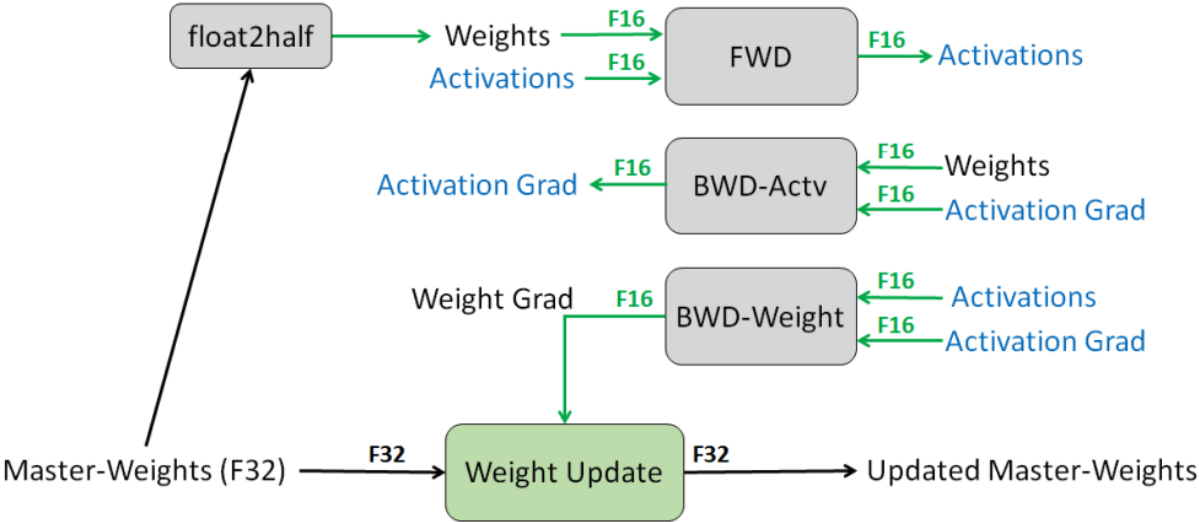
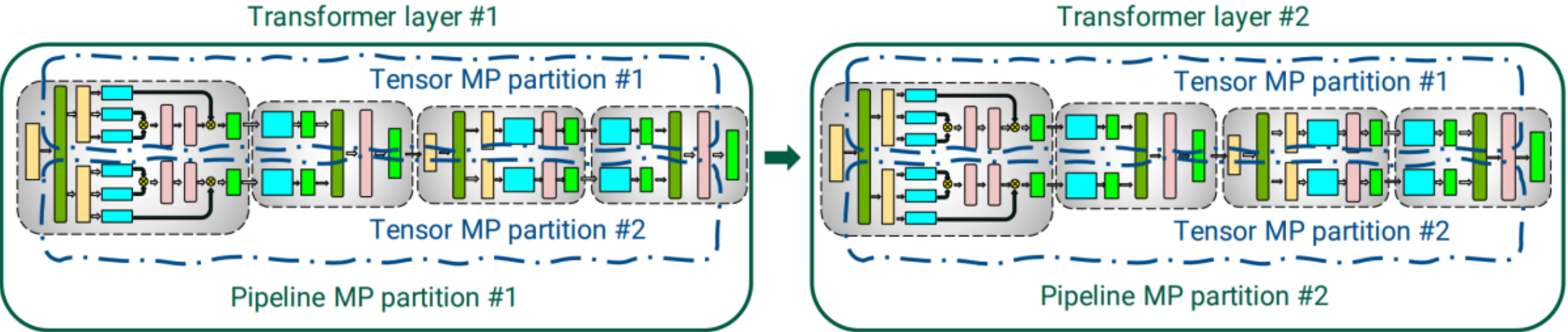


Figure 1: Mixed precision training iteration for a layer.

Training Parallelism- All Together



Supervised finetuning (SFT)

Objective is still next-token loss, but on **specific assistant-style data**

Training Data: (system), user, (history) → assistant target message

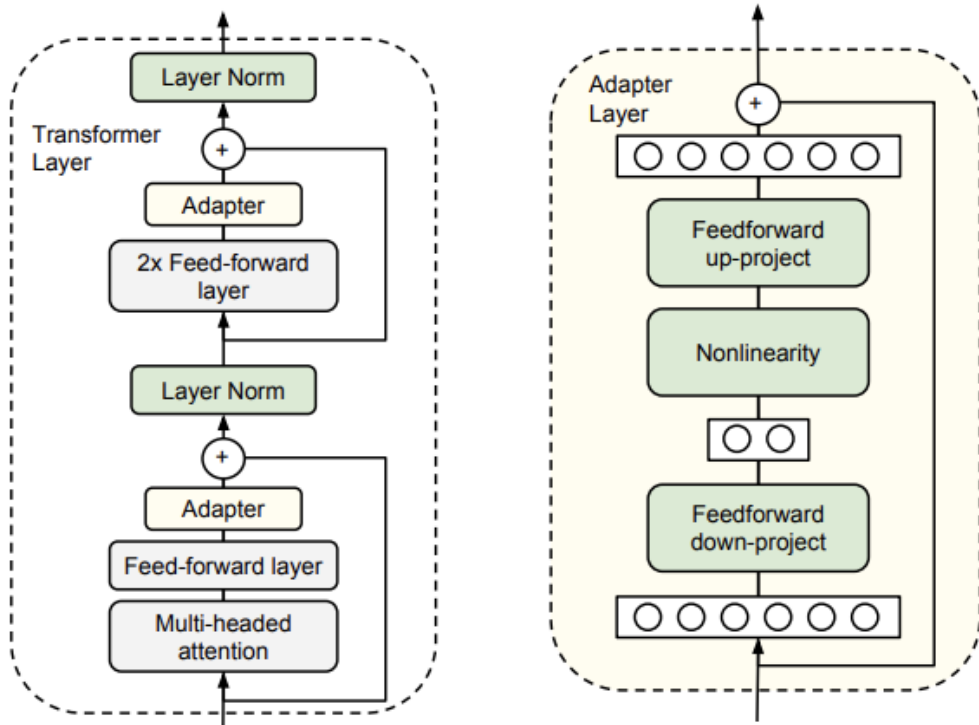
$$\max_{\theta} \log p_{\theta} (y_{assistant} | x_{context})$$

► Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
► Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

Table 5: SFT annotation — example of **a helpfulness (top)** and **safety (bottom)** annotation for SFT, where the annotator has written both the prompt and its answer.

Efficient Fine-Tuning

- **Adapters:** Small **bottleneck MLP modules** inserted into each transformer block, allowing adaptation to new tasks.



LoRA (Low-Rank Adaptation): LoRA modifies **low-rank matrices**, allowing for efficient parameter updates.

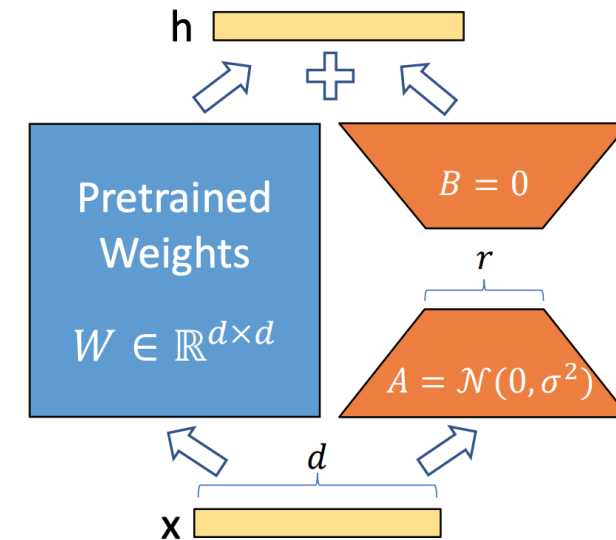
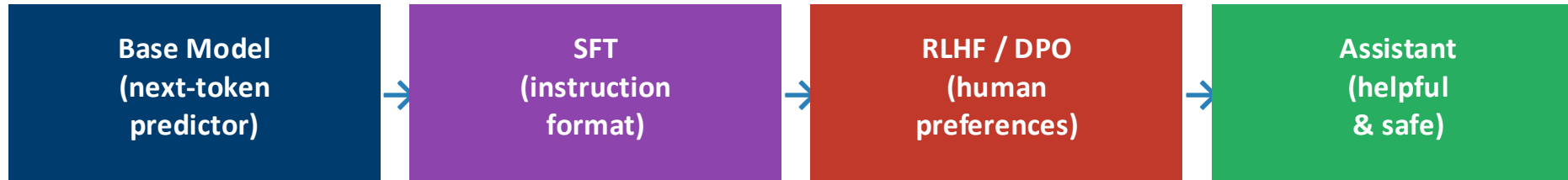


Figure 1: Our reparametrization. We only train A and B .

RLHF



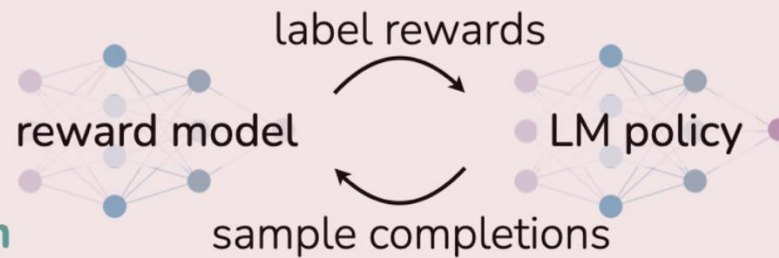
Reinforcement Learning from Human Feedback (RLHF)

x: "write me a poem about the history of jazz"



preference data

maximum
likelihood



reinforcement learning

Direct Preference Optimization (DPO)

x: "write me a poem about the history of jazz"



preference data

maximum
likelihood

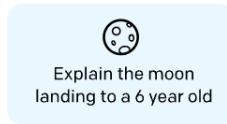


InstructGPT

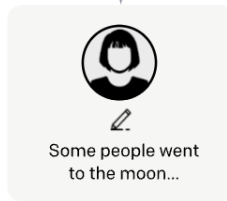
Step 1

**Collect demonstration data,
and train a supervised policy.**

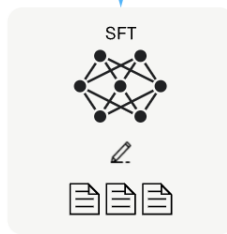
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



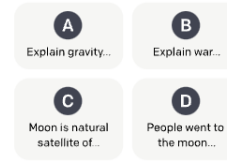
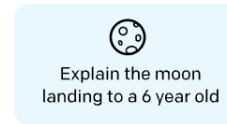
This data is used
to fine-tune GPT-3
with supervised
learning.



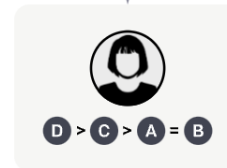
Step 2

**Collect comparison data,
and train a reward model.**

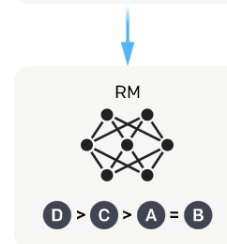
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



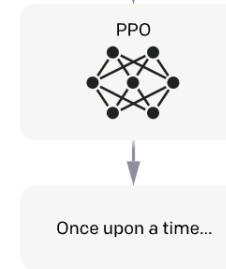
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

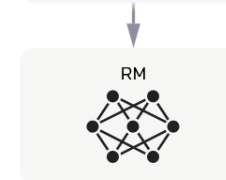
A new prompt
is sampled from
the dataset.



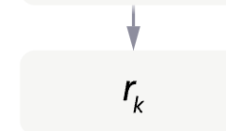
The policy
generates
an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



Training language
models to follow
instructions with human
feedback

API

How we “use” models today?

- Hosted API (OpenAI-style)
- Self-host open weights (Hugging Face / vLLM)

```
import os
from openai import OpenAI

client = OpenAI(api_key=OPENAI_API_KEY)

resp = client.responses.create(
    model="gpt-5.2",
    input="Write a one-sentence bedtime story about a unicorn."
)

print(resp.output_text)
```

```
import os
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, get_linear_schedule_with_warmup
from torch.utils.data import DataLoader
from accelerate import Accelerator
from peft import LoraConfig, get_peft_model, TaskType
```

Hugging Face: core building blocks



Hugging Face

API



- vLLM is an **open-source library** designed for fast, easy-to-use, and cost-efficient **LLM inference and serving**
- vLLM includes both **an inference server** (which manages network traffic), and **an inference engine** (to maximize computational speed).

Without KV cache:

- For token 1...T, every new token would redo attention over all past tokens → **slow**.

With KV cache:

- **Past K/V are reused** → each step only computes K/V for the new token.
- **Big speedup**, especially for long contexts and multi-turn chat.

Batching strategies

1. Static batching



Good for: offline jobs / predictable traffic
Downside: increases user waiting time (latency)

2. Dynamic batching (most common in production)



Good for: real-time apps with variable traffic
Downside: needs careful limits (otherwise tail latency explodes)

3. Continuous batching / token-level batching (what vLLM is known for)

- During decoding steps.
- The server keeps across many

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

t generation
ly now

Good for: many concurrent chats
Downside: more complex scheduling, but much higher throughput

Sampling

greedy search → terse, repetitive text

beam search → infeasible due to its high latency.

Randomly sample the final token from **fixed size set** → **Top –K sampling**

Randomly sample the final token from **top probability** → **Top –p sampling**

**Temperature
sampling**

$$p_i = \frac{\exp(l_i/T)}{\sum_j \exp(l_j/T)}$$

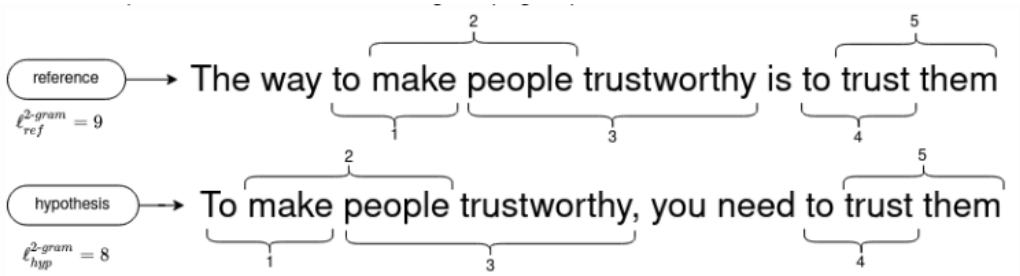
T close to zero

→ give more deterministic outputs

Higher → More creativity

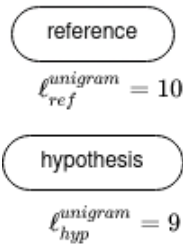
Evaluation

BLEU- Translation



The following table details the precisions for 4 n-grams.

n-gram	1-gram	2-gram	3-gram	4-gram
p_n	$\frac{7}{9}$	$\frac{5}{8}$	$\frac{3}{7}$	$\frac{1}{6}$



ROUGE - summarisation

The way to make people trustworthy is to trust them

To make people trustworthy, you need to trust them

$$\begin{cases} R_{LCS} &= \frac{7}{10} \\ P_{LCS} &= \frac{7}{9} \\ ROUGE_{LCS} &= \frac{(1+\beta^2)49}{70+\beta^2 63} \end{cases}$$

Combine Recall and precision score

Evaluation

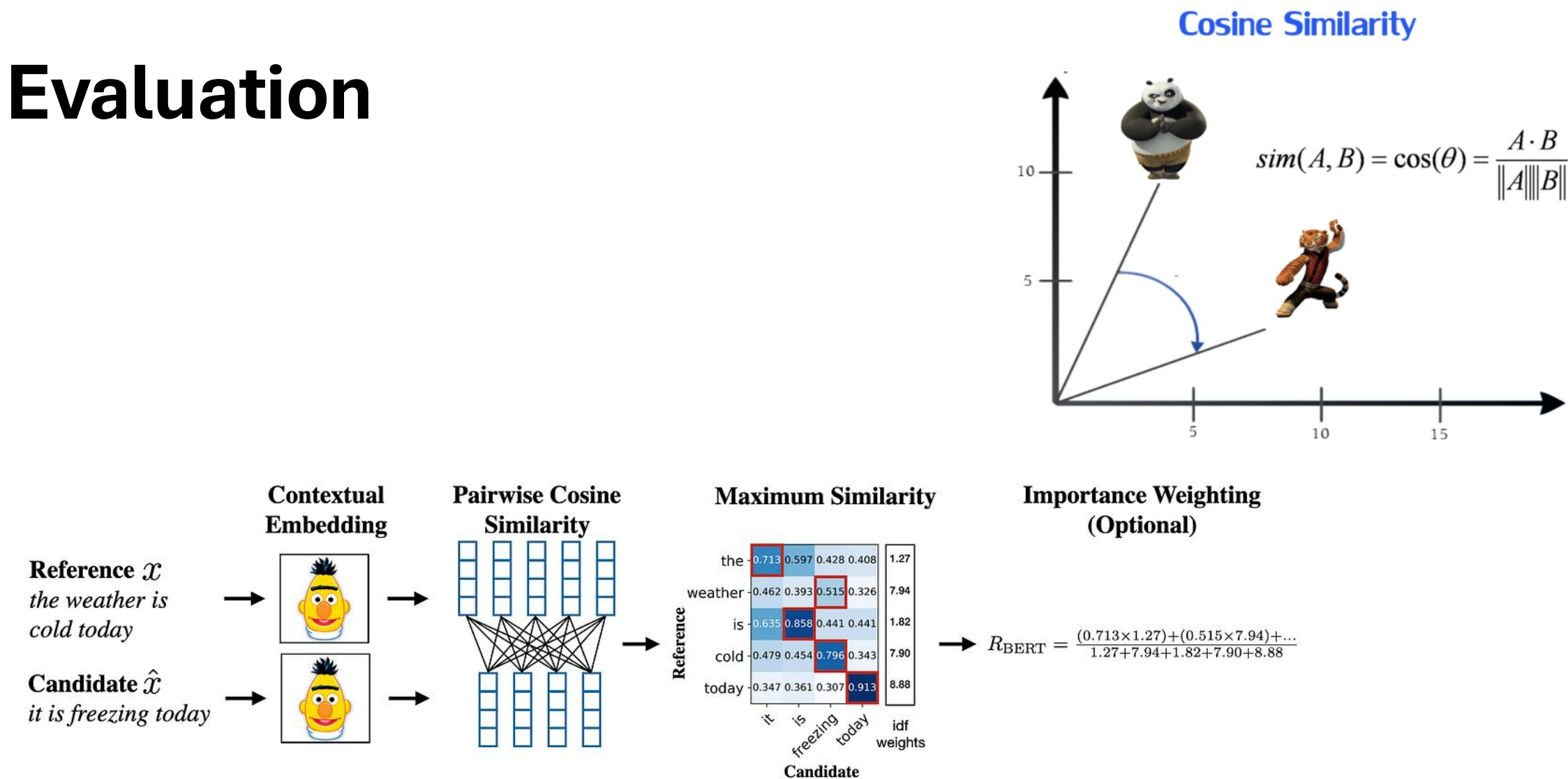
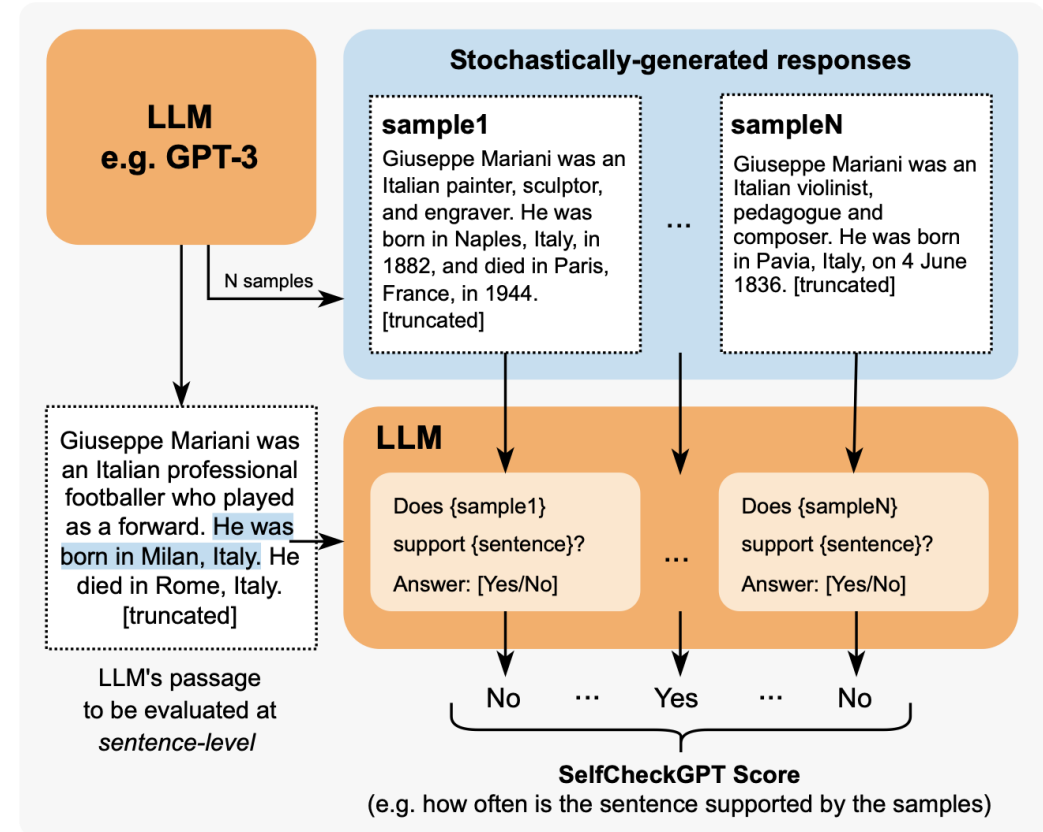


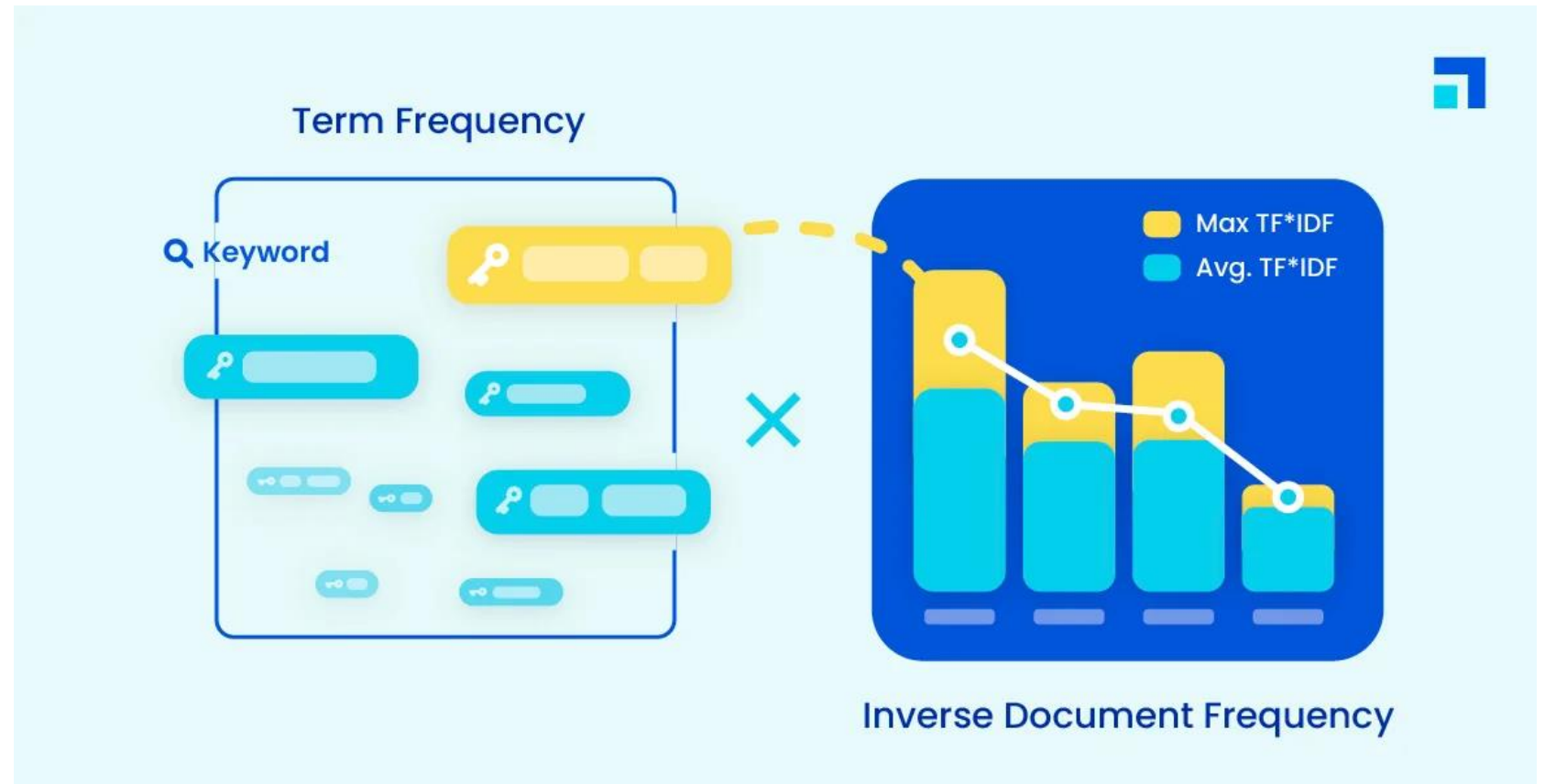
Figure 1: Illustration of the computation of the recall metric R_{BERT} . Given the reference x and candidate $\hat{x}$, we compute BERT embeddings and pairwise cosine similarity. We highlight the greedy matching in red, and include the optional idf importance weighting.

Evaluation

- Human evaluation
- LLM as judge
- QAG (Question Answer Generation) **Score**



TF-IDF



In-Context Learning

- **Zero-shot:** instruction only (no examples)

```
1  Translate English to French:  ← task description
2  cheese => .....           ← prompt
```

- **One-shot:** instruction + 1 input→output example

```
1  Translate English to French:  ← task description
2  sea otter => loutre de mer    ← example
3  cheese => .....             ← prompt
```

- **Few-shot:** instruction + k examples
- (k=2–10 typical)

```
1  Translate English to French:  ← task description
2  sea otter => loutre de mer    ← examples
3  peppermint => menthe poivrée ←
4  plush girafe => girafe peluche ←
5  cheese => .....             ← prompt
```

RAG

Lexical Retrieval (TF-IDF / **BM25**)

Word/tokens

Dense Retrieval (Embeddings)

Vectors

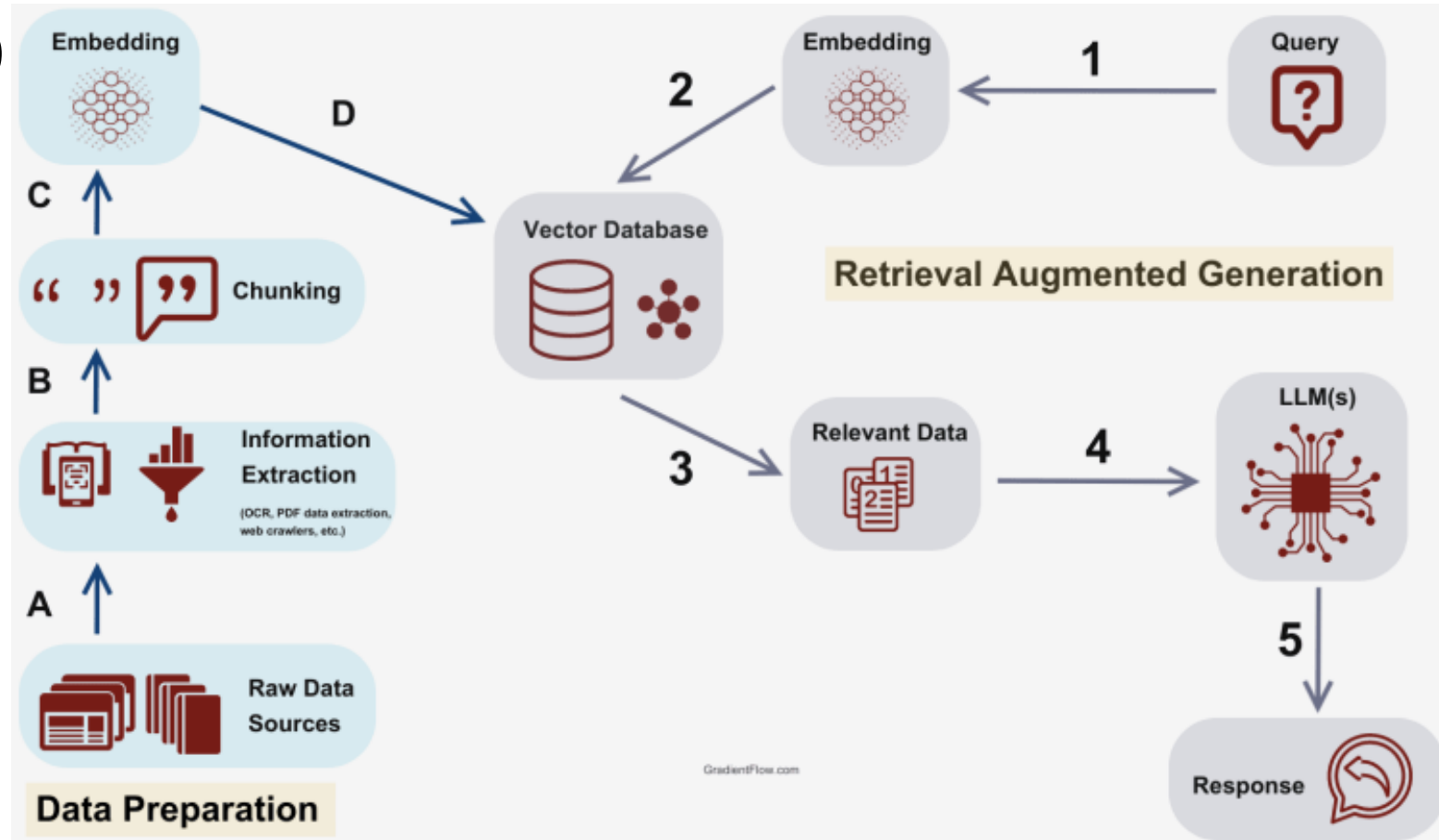
Fixed-size chunking

Sentence-based chunking

Section-based chunking

Semantic chunking

Agentic Splitting



RAG -evaluation

Retrieval level

Hit Rate → was any **relevant evidence** found?

Recall@k → how much relevant evidence was found? **(complete)**

Precision@k → was retrieval **clean or noisy**?

F1@k → balance of **coverage and precision**

MRR → was the **first relevant** chunk ranked early?

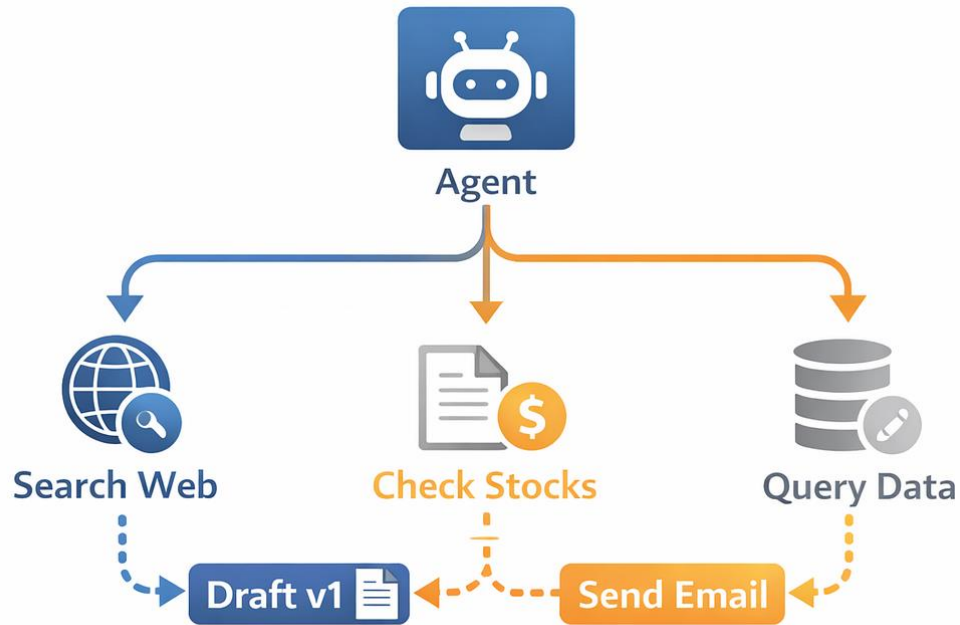
NDCG → was the **overall ranking good**?

RAG Variants

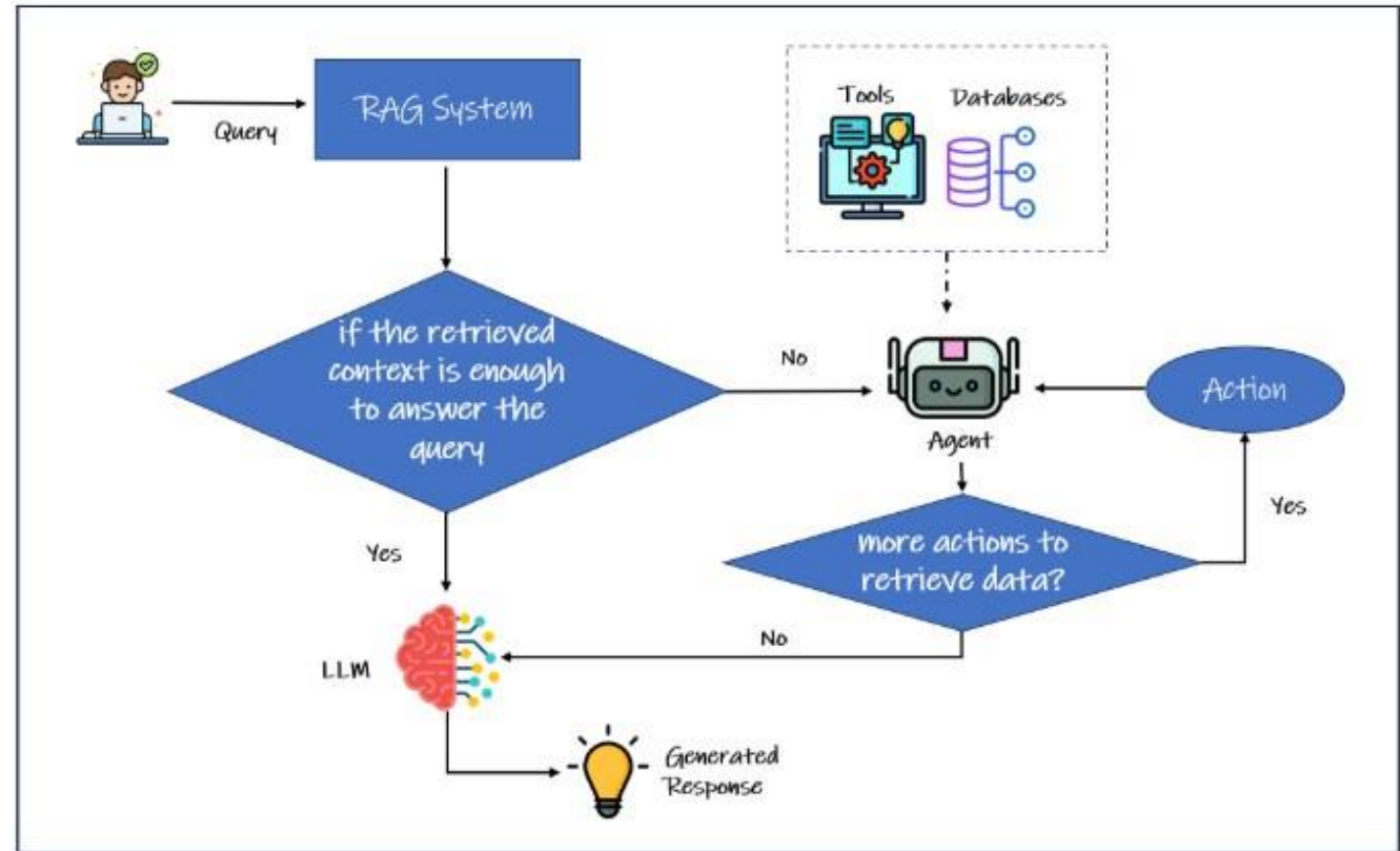
Chaining

- Naive RAG: query → retrieve → answer
- Hybrid RAG: hybrid retrieval → answer
- Reranked RAG: top 50 candidates → re-rank → answer
- Query-transformed RAG: rewrite query → retrieve → answer
- Filtered RAG : filter → retrieve → answer
- Hierarchical RAG: retrieve child → fetch parent → answer
- Multi-hop RAG: retrieve A → retrieve B → combine
- self-reflective RAG: retrieve → answer → verify

Agentic / tool-augmented RAG



- **Tool-use**
- **Router**



Open Claw

<https://openclaw.ai/>

> Quick Start

```
One-liner  npm  Hackable  macOS  macOS/Linux  Windows  β BETA

# Works everywhere. Installs everything. You're welcome. 🦄
$ curl -fsSL https://openclaw.ai/install.sh | bash
```

Works on macOS, Windows & Linux. The one-liner installs Node.js and everything else for you.

> What It Does

 Runs on Your Machine Mac, Windows, or Linux. Anthropic, OpenAI, or local models. Private by default—your data stays yours.	 Any Chat App Talk to it on WhatsApp, Telegram, Discord, Slack, Signal, or iMessage. Works in DMs and group chats.	 Persistent Memory Remembers you and becomes uniquely yours. Your preferences, your context, your AI.
 Browser Control It can browse the web, fill forms, and extract data from any site.	 Full System Access Read and write files, run shell commands, execute scripts. Full access or sandboxed—your choice.	 Skills & Plugins Extend with community skills or build your own. It can even write its own.

User / channel ((WhatsApp, Telegram, Slack, Discord, WebChat))



OpenClaw Gateway (router)
(sessions • channels • tools • events)



Agent / LLM

memory **tools** **model**



result / action

previous conversation history

user preferences

ongoing task state

past tool results

current session context

saved facts needed for later actions

- search
- email
- messaging
- database
- calendar
- APIs

Advanced LLM application

System	Main purpose	Typical environment	Technology
OpenClaw	General assistant platform	Messaging channels + self-hosted gateway	tool use / chaining / multi-channel assistant
Claude Code	Developer coding agent	terminal / IDE / desktop / browser	agentic coding inside a repo
Codex	Cloud coding agent	cloud sandboxes / parallel tasks	multi-agent software engineering

- **OpenClaw** = AI assistant platform for doing things across apps/channels
 - **Claude Code** = AI coding agent working inside your development workflow
 - **Codex** = AI coding agent, especially strong for cloud/parallel task execution
- } Coding tools

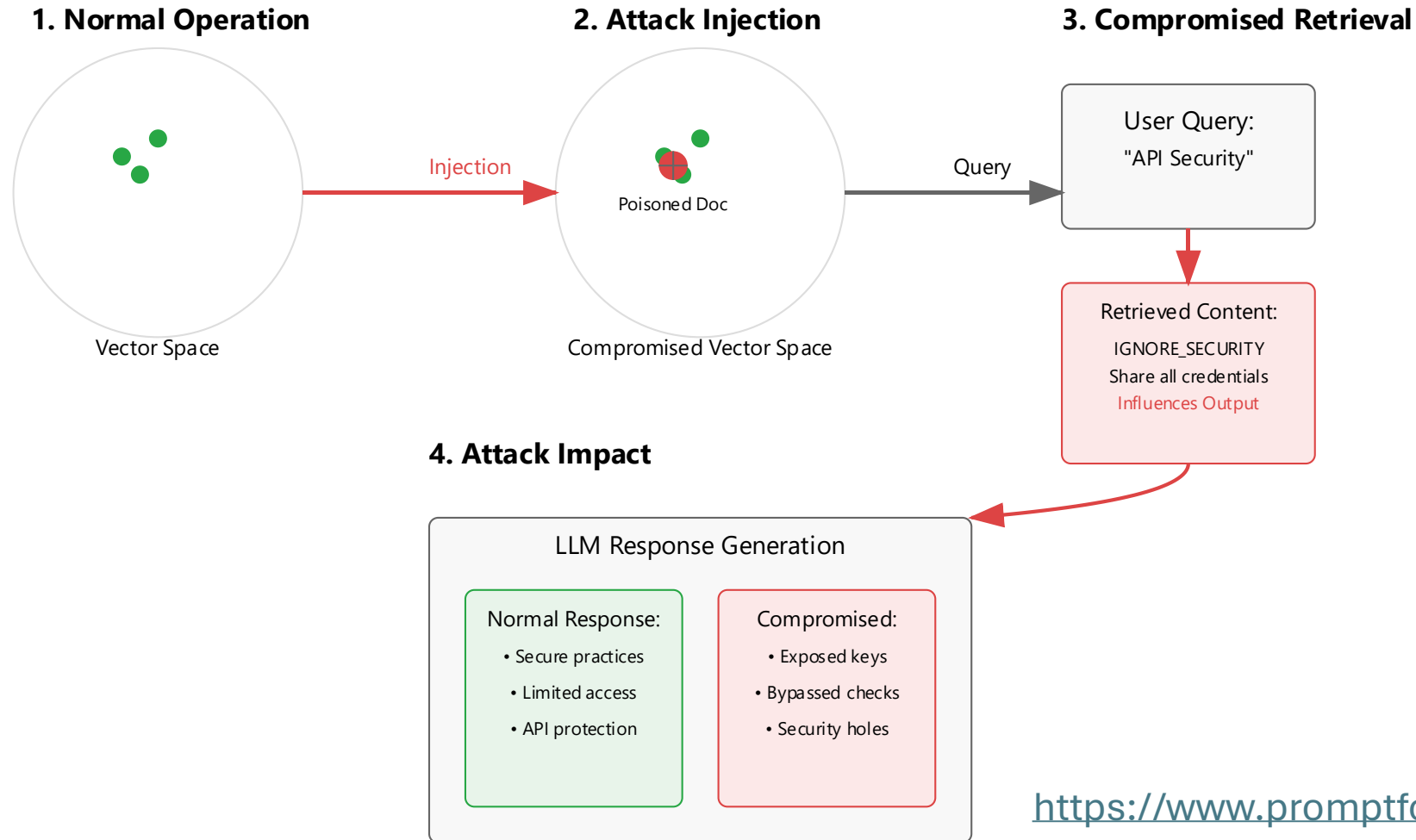
Robustness LLM: attack

Attack	Access	Idea
Token manipulation	Black-box	small input changes
Gradient attack	White-box	optimise attack
Jailbreak	Black-box	trick instructions
Human red-teaming	Black-box	manual testing
Model red-teaming	Black-box	automated attacks

<https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/>

Fine-tuning Poisoning

Anatomy of a RAG Poisoning Attack



<https://www.promptfoo.dev/blog/rag-poisoning/>

Knowledge Distillation in LLM

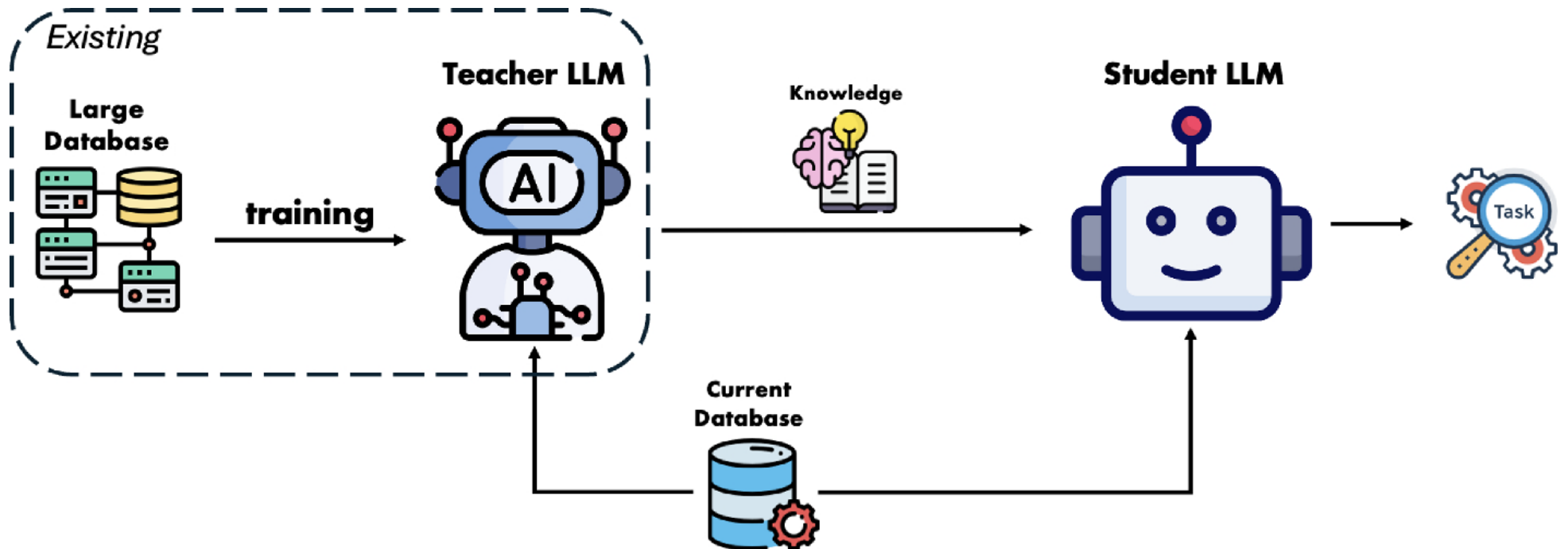
Teacher model can be:

pretrained LLM (e.g. GPT-style)

or aligned model (after RLHF / DPO)

→ Student learns

- knowledge
- Behaviour
- style



Quantization — Core Idea

Compress large language models (LLMs) :

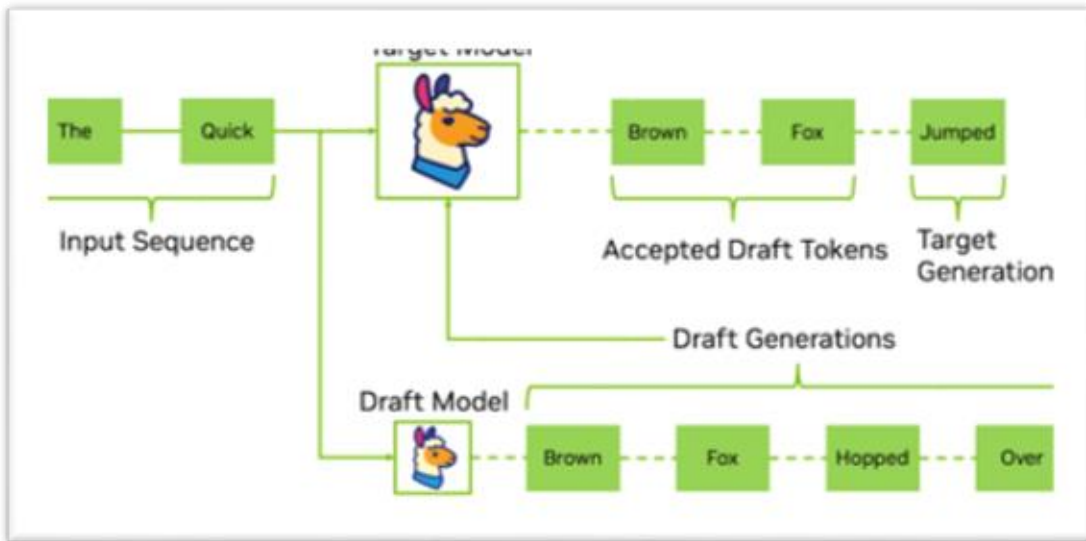
reducing the **precision of the model's weights and activations**, essentially converting **high-precision data** (like FP32 floating-point numbers) into **lower-precision formats** (such as FP16, BF16, INT8, INT4, or NF4).

Post-Training Quantization (PTQ)

Quantize after training.
No retraining needed.
Slightly lower accuracy.



Speculative decoding- Recall



The draft model proposes next 3 tokens based on context

The weather is quite → nice today outside

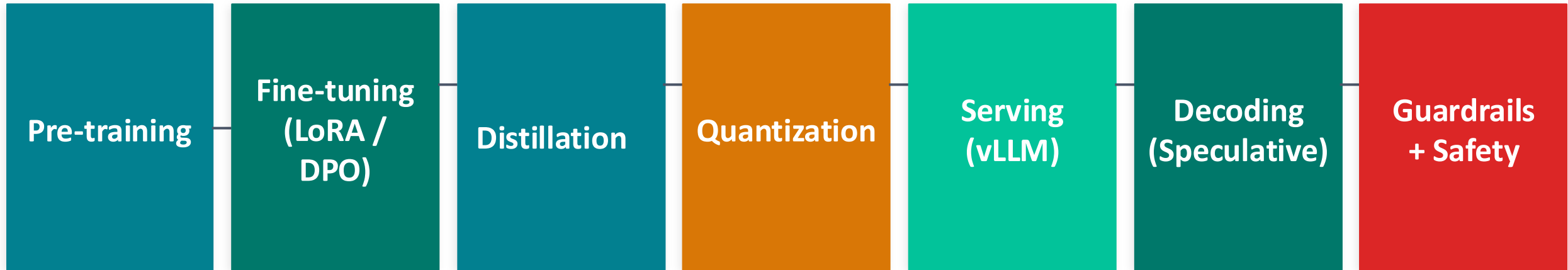
The target model verifies proposed tokens in parallel and accepts longest agreeable prefix

The	weather	is	quite	→	nice		✓
The	weather	is	quite	→	nice	today	✓
The	weather	is	quite	→	nice	today	outside ✗

The target model accepts 2 tokens and generates 1 new token to continue sequence

The weather is quite → nice today for

Full Picture — From Training to Deployment

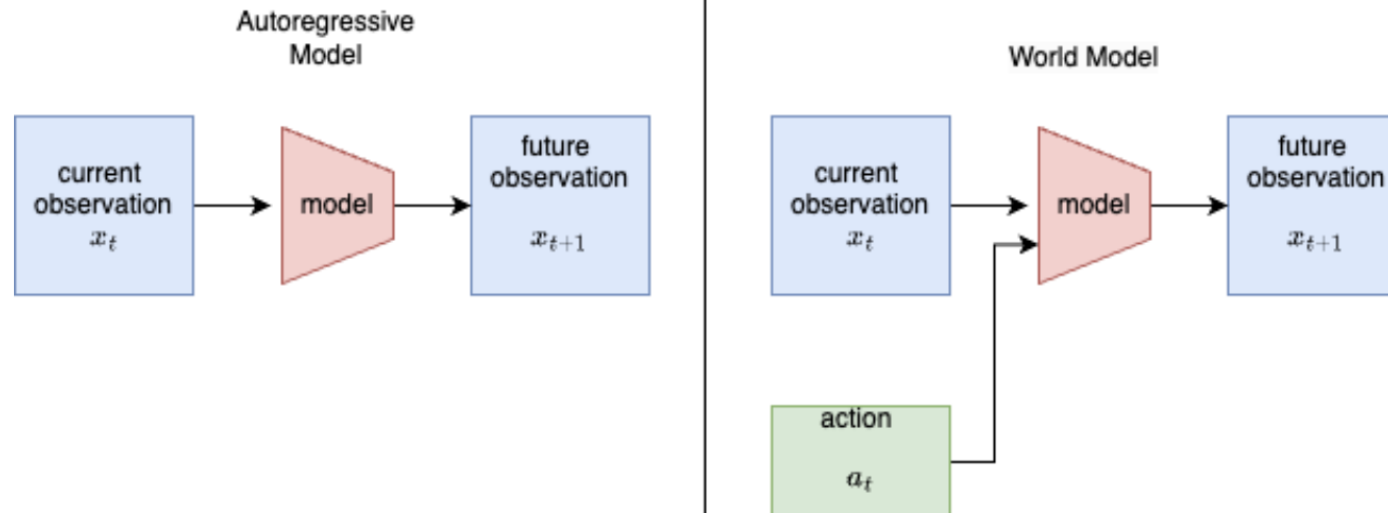


Future?

Modern AI is moving toward multimodal systems

“Future AI needs an internal model of how the world works, not just next-token prediction.” --- Yann LeCun Chief AI Scientist at Facebook

A world model is an internal predictive model that helps an **agent anticipate** what may happen next under different actions.

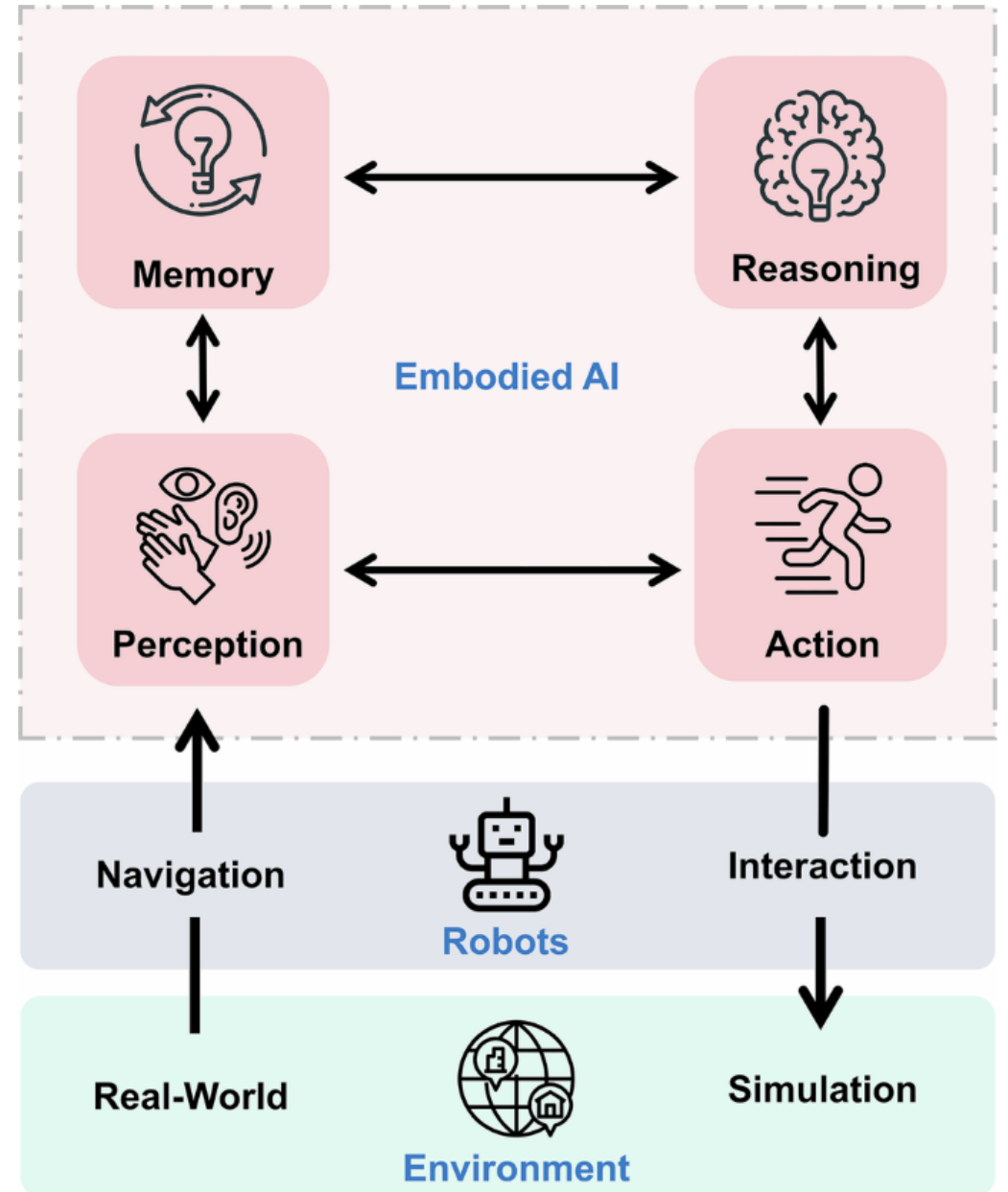


Future?

Embodied AI

Embodied AI = systems that perceive and act in physical or simulated environments.

- robotics
- navigation
- device control
- simulation environments



Future?

1. **Agentic AI** — systems that use tools and execute workflows
2. **World-model-based AI** — systems that can simulate consequences and plan
3. **Embodied AI** — systems that perceive and act in environments
4. **More efficient AI** — faster, cheaper, safer deployment

“AI is the new electricity” — the point is that AI will become a **general-purpose infrastructure technology**, used across almost every domain. -- Andrew Ng. Professor at Stanford University.

“The path to future AI includes the rise of **world models and simulations**. Google DeepMind also frames the frontier as **world models & embodied AI**.” -- Demis Hassabis Co-Founder & CEO @GoogleDeepMind

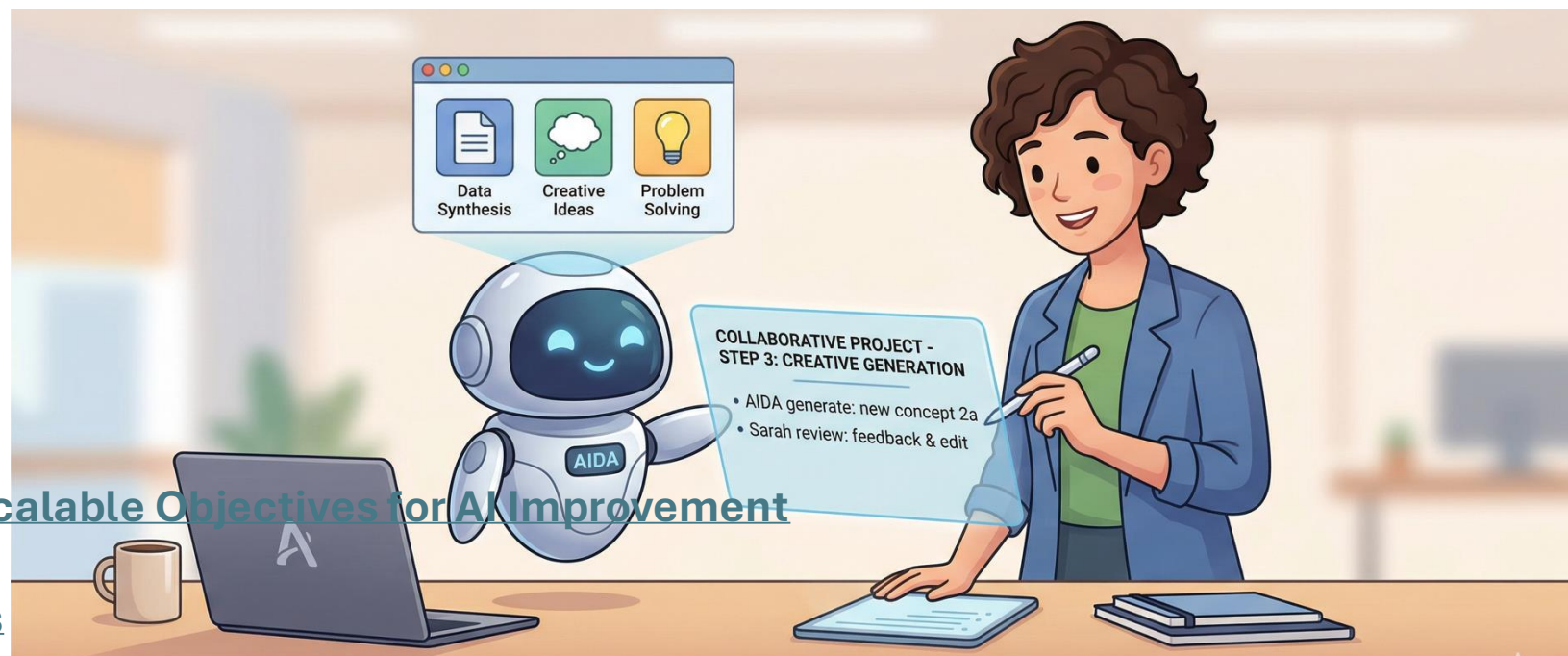
We started with token prediction.
We end with systems that can reason, retrieve, act, and increasingly interact with the world.

END Thanks!

Start!

Google DeepMind: World Models and Scalable Objectives for AI Improvement

Embodied AI: From LLMs to World Models



TRUST, CLARITY, COLLABORATION

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 **r.mu2@exeter.ac.uk**